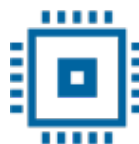




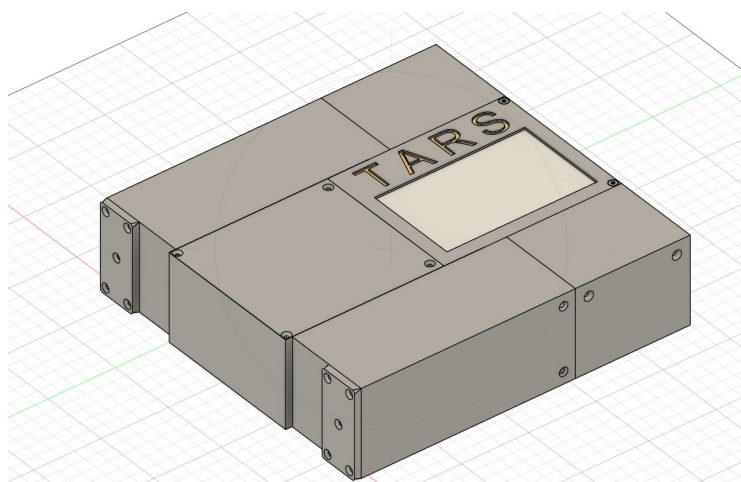
**Departamentul Automatică și
Tehnologia Informației**



**Universitatea
Transilvania
din Brașov**
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

PROIECT LA ROBOTICĂ

Robot TARS



**Păunescu Vlad-Ioan
Pleșu Robert Ștefănuț**

Gr.4LF433 - AIA - II

BRAȘOV, 2024-2025

CUPRINS

Cap 1. DESCRIEREA SCOPULUI ȘI A FUNCȚIONALITĂȚII ROBOTULUI.....	3
1.1 Caracteristici inovatoare	3
1.2 Provocări și soluții	4
1.3 Rezultate	5
Cap. 2. ARHITECTURA.....	6
Cap. 3 CODUL FOLOSIT.....	7
3.1. Servo Abstractor:	7
3.2 TARS Runner	8
3.3. Servo Controller	10
Cap. 4. PLANURI DE VIITOR.....	12
Cap. 5. VIITORUL ROBOTULUI TARS.....	12
Cap. 6. CONCLUZII	13

Cap 1. DESCRIEREA SCOPULUI ȘI A FUNCȚIONALITĂȚII ROBOTULUI

Robotul TARS este un asistent robotic inovator, proiectat să combine utilitatea practică cu o prezență socială agreabilă. Conceput pentru a sprijini utilizatorii în activitățile lor zilnice, TARS aduce un echilibru perfect între funcționalitate și design simplificat.

1.1 Caracteristici inovatoare

Deși la prima vedere TARS poate părea un robot minimalist, cu o structură bazată pe două picioare, fiecare echipat cu o singură articulație, complexitatea sa inginerescă stă într-un mecanism avansat de mobilitate. Pentru a depăși limitările inițiale, echipa de dezvoltare a proiectat un sistem de suspensie unic, imprimat 3D, care a transformat mobilitatea robotului în mod semnificativ.

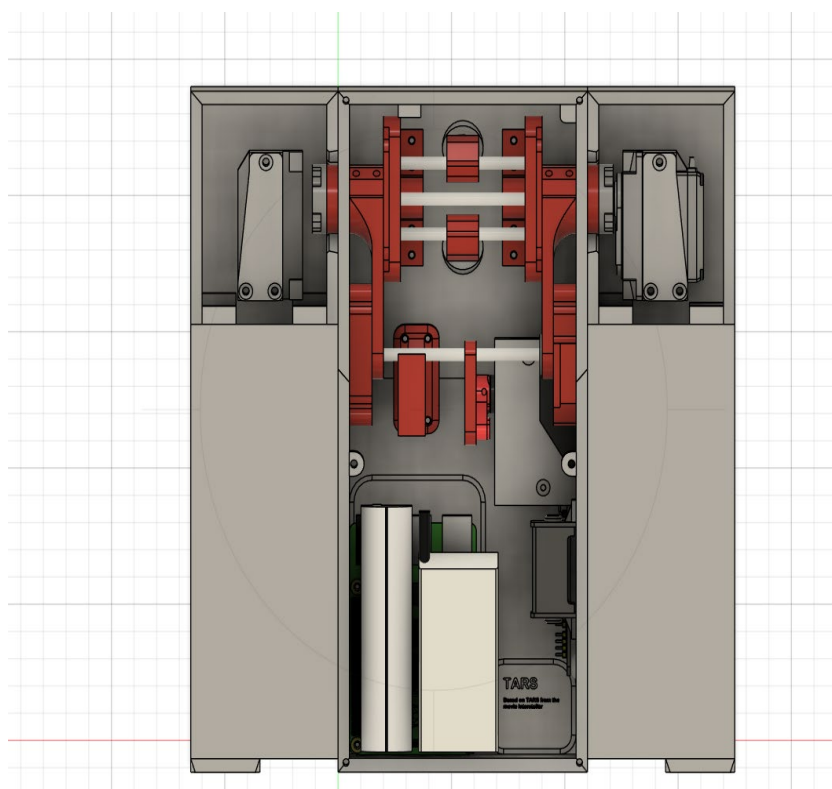


Figura 1 - Asamblu interior

1.2 Provocări și soluții

La primele teste, robotul s-a confruntat cu restricții în mișcare, cauzate de lipsa unui mecanism de flexibilitate între picioare și trunchiul central. Soluția a constat într-un sistem de suspensie integrat, care permite transferul mișcării printr-o punte centrală ridicată, acționată de motoare de mare precizie. Această punte nu doar că permite mișcări naturale ale brațelor, dar oferă și stabilitate crescută pe durata mersului.

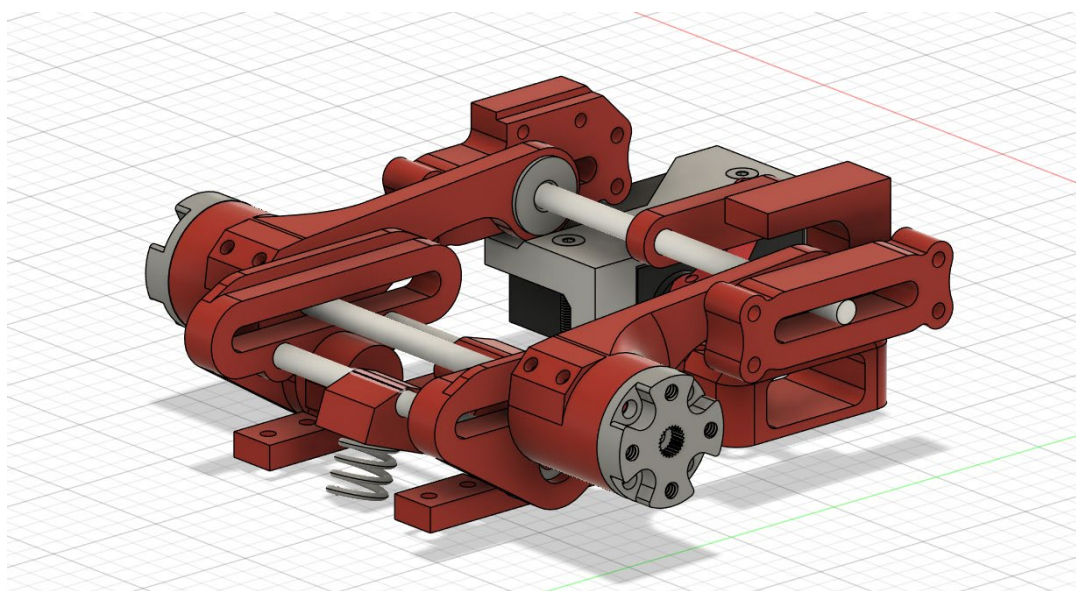


Figura 2 - Sistem de suspensie

Pe partea de software, au apărut dificultăți legate de afișarea pe ecran și de conceperea unui cod pentru apelarea unui sistem de inteligență artificială. Deoarece nu am găsit o soluție AI gratuită potrivită, am analizat alternative și am decis să folosim un controller de Xbox One pentru a controla mișcările robotului. Această abordare a oferit o soluție practică, facilitând controlul robotului în timp ce explorăm opțiuni AI pe termen lung.

1.3 Rezultate

Datorită acestor ajustări, TARS a trecut de la un prototip funcțional, dar instabil, la un model robust și fiabil. Acum, robotul este capabil să execute mișcări continue timp de ore într-un mod natural, replicând fidel mișcările inspirate din celebra producție cinematografică.

Am reușit să conectăm ecranul robotului, afișând deocamdată doar linii de cod. Totodată, robotul răspunde prompt la comenzile transmise prin controllerul de Xbox One, asigurând o interacțiune eficientă și fluidă cu utilizatorul.

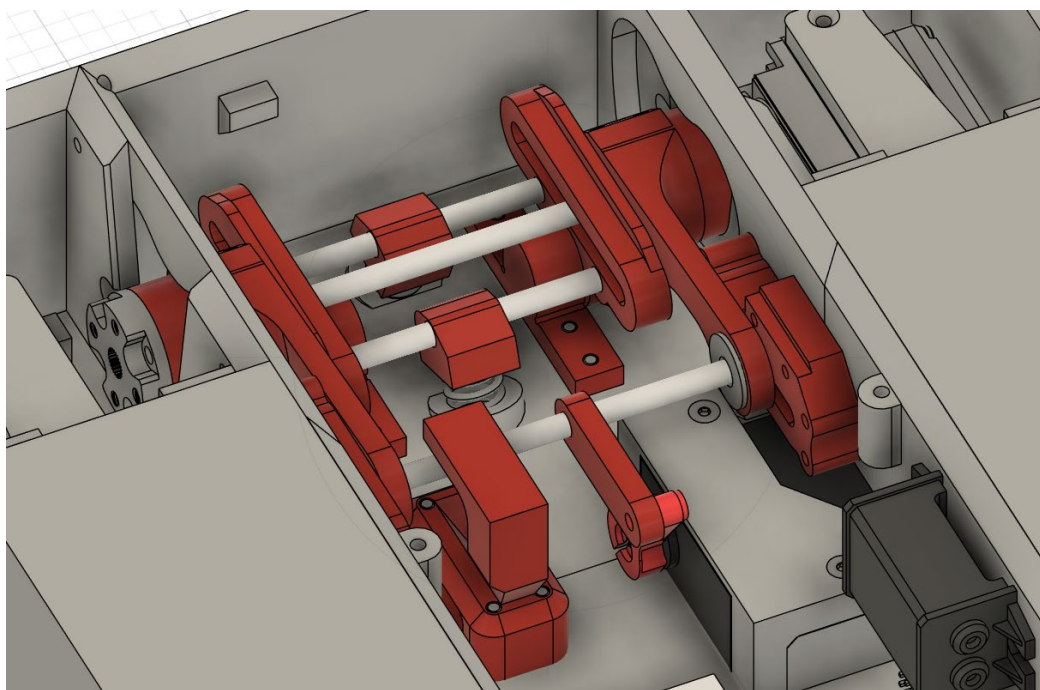
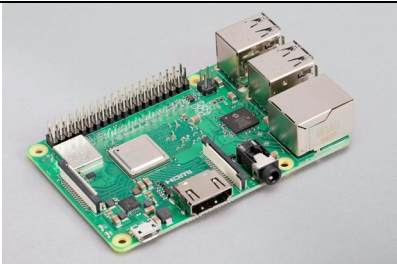
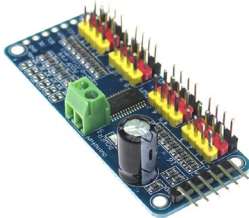
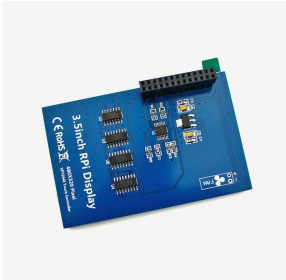


Figura 3 - Sistemul de suspensie in configuratia robotului

Cap. 2. ARHITECTURA

Raspberry Pi Model 3B+	
DC Convertor de Joasa Tensiune 12V-6V	
16-Channel 12-bit PWM/Servo Driver I2C interface PCA9685	
3.5" RPI Display XPT2046	
3x MG996R Servo motor	

Cap. 3 CODUL FOLOSIT

3.1. Servo Abstractor:

```
1  import time
2  import TARS_Servo_Controller3
3
4  def stepForward():
5      TARS_Servo_Controller3.height_neutral_to_up()
6      TARS_Servo_Controller3.torso_neutral_to_forwards()
7      TARS_Servo_Controller3.torso_bump()
8      TARS_Servo_Controller3.torso_return()
9
10 def turnRight():
11     TARS_Servo_Controller3.neutral_to_down()
12     TARS_Servo_Controller3.turn_right()
13     TARS_Servo_Controller3.down_to_neutral()
14     TARS_Servo_Controller3.neutral_from_right()
15
16 def turnLeft():
17     TARS_Servo_Controller3.neutral_to_down()
18     TARS_Servo_Controller3.turn_left()
19     TARS_Servo_Controller3.down_to_neutral()
20     TARS_Servo_Controller3.neutral_from_left()
21
22 def pose():
23     TARS_Servo_Controller3.neutral_to_down()
24     TARS_Servo_Controller3.torso_neutral_to_backwards()
25     TARS_Servo_Controller3.down_to_up()
26
27 def unpose():
28     TARS_Servo_Controller3.torso_return2()
29
30
```

3.2 TARS Runner

```
1  import evdev
2  import time
3  import TARS_Servo_Abstractor3
4  import TARS_Servo_Controller3
5  from evdev import InputDevice, categorize, ecodes
6  import Adafruit_PCA9685
7
8  pwm = Adafruit_PCA9685.PCA9685()
9
10 #port
11 pwm.set_pwm(3, 3, 610)
12 pwm.set_pwm(4, 4, 570)
13 pwm.set_pwm(5, 5, 570)
14 #starboard
15 pwm.set_pwm(6, 6, 200)
16 pwm.set_pwm(7, 7, 200)
17 pwm.set_pwm(8, 8, 240)
18
19 # Set frequency to 60hz, good for servos.
20 pwm.set_pwm_freq(60)
21
22 gamepad = InputDevice('/dev/input/event3')
23
24 lTrg = 37
25 rTrg = 50
26 upBtn = 46
27 downBtn = 32
28 lBtn = 18
29 rBtn = 33
30 xBtn = 23
31 yBtn = 35
32 aBtn = 36
33 bBtn = 34
34 minusBtn = 49
35 plusBtn = 24
36
37 toggle = True
38 pose = False
39
40 print(gamepad)
```

```
1  for event in gamepad.read_loop():
2      if event.type == ecodes.EV_KEY:
3          if event.value == 1:
4              if event.code == lTrg:
5                  print("Left Trigger")
6                  if toggle == True:
7                      TARS_Servo_Controller3.portMainPlus()
8                  elif toggle == False:
9                      TARS_Servo_Controller3.portMainMinus()
10             elif event.code == rTrg:
11                 print("Right Trigger")
12                 if toggle == True:
13                     TARS_Servo_Controller3.starMainPlus()
14                 elif toggle == False:
15                     TARS_Servo_Controller3.starMainMinus()
16             elif event.code == upBtn:
17                 print("Up - Step Forward")
18                 TARS_Servo_Abstractor3.stepForward()
19             elif event.code == downBtn:
20                 print("Down")
21                 if pose == False:
22                     TARS_Servo_Abstractor3.pose()
23                     pose = True
24                 elif pose == True:
25                     TARS_Servo_Abstractor3.unpose()
26                     pose = False
27             elif event.code == lBtn:
28                 print("Left - Turn Left")
29                 TARS_Servo_Abstractor3.turnLeft()
30             elif event.code == rBtn:
31                 print("Right - Turn Right")
32                 TARS_Servo_Abstractor3.turnRight()
33             elif event.code == xBtn:
34                 print("X")
35                 if toggle == True:
36                     TARS_Servo_Controller3.starForarmPlus()
37                 elif toggle == False:
38                     TARS_Servo_Controller3.starForarmMinus()
39             elif event.code == yBtn:
40                 print("Y")
41                 if toggle == True:
42                     TARS_Servo_Controller3.portForarmPlus()
43                 elif toggle == False:
44                     TARS_Servo_Controller3.portForarmMinus()
45             elif event.code == aBtn:
46                 print("A")
47                 if toggle == True:
48                     TARS_Servo_Controller3.starHandPlus()
49                 elif toggle == False:
50                     TARS_Servo_Controller3.starHandMinus()
51             elif event.code == bBtn:
52                 print("B")
53                 if toggle == True:
54                     TARS_Servo_Controller3.portHandPlus()
55                 elif toggle == False:
56                     TARS_Servo_Controller3.portHandMinus()
57             elif event.code == plusBtn:
58                 print("+")
59                 toggle = True
60             elif event.code == minusBtn:
61                 print("-")
62                 toggle = False
63         elif event.value == 0:
64             print("Stop")
```

3.3. Servo Controller

```
1  import evdev
2  import time
3  import TARS_Servo_Abstractor3
4  import TARS_Servo_Controller3
5  from evdev import InputDevice, categorize, ecodes
6  import Adafruit_PCA9685
7
8  pwm = Adafruit_PCA9685.PCA9685()
9
10 #port
11 pwm.set_pwm(3, 3, 610)
12 pwm.set_pwm(4, 4, 570)
13 pwm.set_pwm(5, 5, 570)
14 #starboard
15 pwm.set_pwm(6, 6, 200)
16 pwm.set_pwm(7, 7, 200)
17 pwm.set_pwm(8, 8, 240)
18
19 # Set frequency to 60hz, good for servos.
20 pwm.set_pwm_freq(60)
21
22 gamepad = InputDevice('/dev/input/event3')
23
24 lTrg = 37
25 rTrg = 50
26 upBtn = 46
27 downBtn = 32
28 lBtn = 18
29 rBtn = 33
30 xBtn = 23
31 yBtn = 35
32 aBtn = 36
33 bBtn = 34
34 minusBtn = 49
35 plusBtn = 24
36
37 toggle = True
38 pose = False
39
40 print(gamepad)
```

```
1  for event in gamepad.read_loop():
2      if event.type == ecodes.EV_KEY:
3          if event.value == 1:
4              if event.code == lTrg:
5                  print("Left Trigger")
6                  if toggle == True:
7                      TARS_Servo_Controller3.portMainPlus()
8                  elif toggle == False:
9                      TARS_Servo_Controller3.portMainMinus()
10             elif event.code == rTrg:
11                 print("Right Trigger")
12                 if toggle == True:
13                     TARS_Servo_Controller3.starMainPlus()
14                 elif toggle == False:
15                     TARS_Servo_Controller3.starMainMinus()
16             elif event.code == upBtn:
17                 print("Up - Step Forward")
18                 TARS_Servo_Abstractor3.stepForward()
19             elif event.code == downBtn:
20                 print("Down")
21                 if pose == False:
22                     TARS_Servo_Abstractor3.pose()
23                     pose = True
24                 elif pose == True:
25                     TARS_Servo_Abstractor3.unpose()
26                     pose = False
27             elif event.code == lBtn:
28                 print("Left - Turn Left")
29                 TARS_Servo_Abstractor3.turnLeft()
30             elif event.code == rBtn:
31                 print("Right - Turn Right")
32                 TARS_Servo_Abstractor3.turnRight()
33             elif event.code == xBtn:
34                 print("X")
35                 if toggle == True:
36                     TARS_Servo_Controller3.starForarmPlus()
37                 elif toggle == False:
38                     TARS_Servo_Controller3.starForarmMinus()
39             elif event.code == yBtn:
40                 print("Y")
41                 if toggle == True:
42                     TARS_Servo_Controller3.portForarmPlus()
43                 elif toggle == False:
44                     TARS_Servo_Controller3.portForarmMinus()
45             elif event.code == aBtn:
46                 print("A")
47                 if toggle == True:
48                     TARS_Servo_Controller3.starHandPlus()
49                 elif toggle == False:
50                     TARS_Servo_Controller3.starHandMinus()
51             elif event.code == bBtn:
52                 print("B")
53                 if toggle == True:
54                     TARS_Servo_Controller3.portHandPlus()
55                 elif toggle == False:
56                     TARS_Servo_Controller3.portHandMinus()
57             elif event.code == plusBtn:
58                 print("+")
59                 toggle = True
60             elif event.code == minusBtn:
61                 print("-")
62                 toggle = False
63         elif event.value == 0:
64             print("Stop")
```

Cap. 4. PLANURI DE VIITOR

Un obiectiv esențial în dezvoltarea viitoare a robotului TARS este integrarea comenzilor vocale, susținută de tehnologii avansate de inteligență artificială. Această funcționalitate va permite utilizatorilor să comunice direct cu robotul, care va interpreta și executa comenzi în timp real, devenind astfel un asistent interactiv și personalizat.

Totodată, securitatea și confidențialitatea datelor sunt priorități majore. Pentru a proteja informațiile utilizatorilor, TARS va procesa local toate interacțiunile, evitând transmiterea datelor către servere externe și respectând normele GDPR. În plus, vor fi implementate funcții pentru gestionarea și ștergerea datelor, oferind utilizatorilor control deplin asupra informațiilor personale.

Progresul proiectului va fi susținut prin colaborări strategice cu organizații de cercetare, universități și companii din domeniul roboticii și AI. Aceste parteneriate vor facilita integrarea unor tehnologii de ultimă generație și testarea robotului în diverse medii controlate, pentru a asigura performanțe optime.

Cap. 5. VIITORUL ROBOTULUI TARS

Pe lângă integrarea comenzilor vocale și a inteligenței artificiale, TARS are un potențial vast de extindere, vizând multiple direcții de dezvoltare:

- **Design modular:** Robotul va fi îmbunătățit atât estetic, cât și funcțional, prin adoptarea unui design modular, ce va permite instalarea facilă a noilor componente.
- **Autonomie sporită:** Integrarea unor tehnologii avansate de învățare automată îi va permite robotului să opereze independent și să se adapteze la schimbările din mediul înconjurător.
- **Integrare smart home:** TARS va putea comunica cu dispozitive inteligente din locuință, precum sisteme de iluminat, termostate sau electrocasnice, oferind o experiență completă de automatizare.
- **Siguranța locuinței:** Integrarea cu sisteme de securitate va permite monitorizarea casei și notificarea utilizatorilor sau autorităților în cazul unor situații de urgență.
- **Interacțiune socială:** TARS va deveni un companion social, sprijinind utilizatorii care se confruntă cu izolarea și oferind activități recreative și conversație.

Cap. 6. CONCLUZII

Robotul TARS reprezintă un pas important în evoluția asistenților robotizați destinați să sprijine utilizatorii în diverse activități cotidiene. Cu un design simplu, dar eficient, TARS oferă deja un prototip solid care poate fi îmbunătățit prin integrarea de noi tehnologii, precum comenzi vocale și AI. În viitor, TARS va deveni nu doar un instrument de asistență, ci și un companion de încredere, capabil să răspundă nevoilor diverse ale utilizatorilor și să adreseze problemele sociale și de accesibilitate.

